GMRes: implementación con Arnoldi y rotaciones de Givens

Clase 14 · Capítulo 5 · Secciones 5.9.4–5.9.5

Análisis Numérico (LMA) · Métodos Numéricos II (LCD)

2 de octubre de 2026

Facultad de Ingeniería Química · UNL

Dónde quedamos

GMRes

$$x_k = \operatorname*{argmin}_{x \in x_0 + \mathcal{K}_k} \|b - Ax\|_2, \qquad \mathcal{K}_k = \operatorname{span}\{r_0, Ar_0, \ldots, A^{k-1} r_0\}.$$

- $\|r_k\|_2 = \min_{p \in \overline{\mathcal{P}}_k} \|p(A) r_0\|_2$; terminación en $\leq n$ pasos; si $\|I - A\|_2 \leq \rho < 1$, $\|r_k\|_2 \leq \rho^k \|r_0\|_2$.
- Precondicionado a izquierda o a derecha, sin tocar el algoritmo.
- En el ejemplo con convección: 16–19 iteraciones independientes de n.

La pregunta pendiente

¿Cómo se calcula ese argmin? Es un problema de mínimos cuadrados con n ecuaciones y k incógnitas, distinto en cada iteración.

Meta: que cada iteración cueste $O(nk)$, y que el residuo se conozca sin calcular x_k.

Plan de la clase

1. **5.9.4 Arnoldi:** una base ortonormal de $\mathcal{K}_k$, la relación $AV_k = V_{k+1}H_k$, y la reducción a un problema de mínimos cuadrados chico.
2. **Breakdown:** cuándo Arnoldi se detiene, y por qué eso es una buena noticia.
3. **Gram–Schmidt clásico vs. modificado:** la estabilidad importa.
4. **Restart:** cuando la memoria no alcanza.
5. **5.9.5 Rotaciones de Givens:** actualizar la factorización QR de H_k con una rotación por iteración. El algoritmo final.
6. **Números y ejercicios.**

5.9.4 El proceso de Arnoldi

Reducir el problema a k incógnitas

Supongamos que tenemos una base ortonormal $\{v_1, \ldots, v_k\}$ de $\mathcal{K}_k$, y sea

$$V_k = \begin{bmatrix} v_1 & v_2 & \cdots & v_k \end{bmatrix} \in \mathbb{R}^{n \times k}.$$

Entonces $z \in \mathcal{K}_k \iff z = V_k y$ para un único $y \in \mathbb{R}^k$, y

$$\|b - A(x_0 + V_k y)\|_2 = \|r_0 - AV_k y\|_2.$$

GMRes, reformulado

$$\text{minimizar} \quad \|r_0 - AV_k\, y\|_2, \quad y \in \mathbb{R}^k, \qquad \text{y luego} \qquad x_k = x_0 + V_k y_k.$$

Es un problema de mínimos cuadrados estándar (QR de AV_k). Pero requiere formar AV_k, una matriz de $n \times k$, en cada iteración.

La clave va a ser elegir la base de manera que AV_k tenga una estructura muy simple.

Arnoldi = Gram–Schmidt aplicado al espacio de Krylov

Algoritmo de Arnoldi

Dados A, x_0, b, k.

$r_0 = b - Ax_0$; $\quad v_1 = r_0/\|r_0\|_2$;

para $i = 1, \ldots, k-1$

$\quad w = Av_i$;

$\quad z = w - \sum_{j=1}^{i} (w^T v_j)\, v_j$; % quitar las componentes en $v_1, \ldots, v_i$

$\quad v_{i+1} = z/\|z\|_2$;

fin

- En lugar de ortogonalizar $r_0, Ar_0, A^2 r_0, \ldots$ (que tienden a ser casi paralelos), se ortogonaliza Av_i: misma base, mucho más estable.
- $Av_i \in \mathcal{K}_{i+1}$, así que $v_{i+1} \in \mathcal{K}_{i+1}$ y $\text{span}\{v_1, \ldots, v_{i+1}\} = \mathcal{K}_{i+1}$.
- Usa A solo a través de productos $\xi \mapsto A\xi$: un mat-vec por paso.

¿Y si $z = 0$? Entonces $Av_i \in \mathcal{K}_i$ y la dimensión no aumenta: el algoritmo no puede seguir (*breakdown*).

El breakdown es una buena noticia

Lema (5.43)

Sea A no singular y sea i el menor entero tal que $z_{i+1} = Av_i - \sum_{j=1}^{i} \left((Av_i)^T v_j\right) v_j = 0$. Entonces $x_* = A^{-1}b \in x_0 + \mathcal{K}_i$.

Demostración. Como $Av_j \in \mathcal{K}_{j+1}$ para $j < i$ y, por hipótesis, $Av_i \in \mathcal{K}_i$, resulta $A\mathcal{K}_i \subset \mathcal{K}_i$. Con $V_i = [v_1 \cdots v_i]$:

$$AV_i = V_i H, \qquad H \in \mathbb{R}^{i \times i} \text{ no singular (pues } A \text{ lo es).}$$

Si $x_i - x_0 = V_i y$ y $\beta = \|r_0\|_2$, como $r_0 = \beta V_i e_1$ y V_i tiene columnas ortonormales,

$$\|r_i\|_2 = \|\beta V_i e_1 - AV_i y\|_2 = \|V_i(\beta e_1 - Hy)\|_2 = \|\beta e_1 - Hy\|_2,$$

que se anula con $y = \beta H^{-1} e_1$. ■

Lectura

Si Arnoldi se detiene en el paso i, GMRes ya encontró la solución exacta. Se lo llama *breakdown afortunado* (*lucky breakdown*).

La relación de Arnoldi

Definimos los coeficientes que calcula Arnoldi:

$$h_{ji} := (Av_i)^T v_j \quad (j \leq i), \qquad h_{i+1,i} := \|z_{i+1}\|_2.$$

Despejando Av_i en el algoritmo:

$$Av_i = \sum_{j=1}^{i+1} h_{ji}\, v_j \qquad \Longrightarrow \qquad \underbrace{A}_{n\times n}\,\underbrace{V_k}_{n\times k} = \underbrace{V_{k+1}}_{n\times(k+1)}\,\underbrace{H_k}_{(k+1)\times k}$$

H_k **es Hessenberg superior**: $h_{ji} = 0$ si $j > i+1$, porque Av_i solo tiene componentes en $v_1, \ldots, v_{i+1}$.

Y crece de a una columna: H_{k+1} es H_k con una columna nueva y una fila de ceros más (salvo el último elemento).

$$H_4 = \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix} \in \mathbb{R}^{5\times 4}$$

El problema chico

Con $\beta = \|r_0\|_2$, $r_0 = \beta v_1 = \beta V_{k+1} e_1$, y la relación de Arnoldi:

$$r_k = r_0 - AV_k y = \beta V_{k+1} e_1 - V_{k+1} H_k y = V_{k+1}\big(\beta e_1 - H_k y\big).$$

Como V_{k+1} tiene columnas ortonormales,

$$\boxed{\|r_k\|_2 = \|\beta e_1 - H_k\, y\|_2}$$

Ganancia

- El problema de mínimos cuadrados pasó de $n \times k$ a $(k+1) \times k$.
- El residuo se conoce sin calcular x_k ni $b - Ax_k$.
- $x_k = x_0 + V_k y_k$ hace falta solo al final.

GMRes, versión preliminar 1

GMRes (preliminar 1)

Dados A, x_0, b, $\mathrm{tol} > 0$, $k_{\text{máx}} \in \mathbb{N}$.

$r_0 = b - Ax_0$; $\quad \beta = \rho = \|r_0\|_2$; $\quad v_1 = r_0/\rho$; $\quad k = 0$;

mientras $\rho > \mathrm{tol}\,\|b\|_2$ **y** $k < k_{\text{máx}}$

$k = k + 1$; $\quad w = Av_k$;

para $j = 1, \ldots, k$: $\quad h_{jk} = w^T v_j$;

$v_{k+1} = w - \sum_{j=1}^{k} h_{jk} v_j$; % (*!) Gram–Schmidt clásico

$h_{k+1,k} = \|v_{k+1}\|_2$; $\quad v_{k+1} = v_{k+1}/h_{k+1,k}$;

$y_k = \operatorname{argmin}_{y \in \mathbb{R}^k} \|\beta e_1 - H_k y\|_2$; % (*)

$\rho = \|\beta e_1 - H_k y_k\|_2$; % (**)

fin

$x_k = x_0 + V_k y_k$.

- Por iteración: un mat-vec y k productos escalares para ortogonalizar: $O(nk)$.
- (*)–(**): un problema de $(k+1) \times k$, por ejemplo con $\backslash$ en $O(k^3)$. Aceptable si $k \ll n$, pero se puede hacer en $O(k)$: es lo que haremos con Givens.

Memoria y restart

El costo de no tener recurrencia corta

Para hacer k iteraciones hay que guardar $k+1$ vectores de largo n, y la ortogonalización de la iteración k cuesta $O(nk)$: el total hasta la iteración k es $O(nk^2)$.

GMRes con restart, GMRes(m)

1. Hacer a lo sumo m iteraciones (con m lo que permita la memoria).
2. Si no se alcanzó la tolerancia, reiniciar desde $x_0 \leftarrow x_m$, descartando la base.

- Memoria fija: $m+1$ vectores.
- No hay teoría de convergencia para GMRes(m), y puede ser mucho más lento (o estancarse). Cuando funciona —a menudo— el ahorro es grande.
- Ver los números al final de la clase.

Gram–Schmidt clásico vs. modificado

Clásico (CGS)

$z = w - \sum_{j=1}^{k} (w^T v_j)\, v_j$

Todos los coeficientes se calculan con el *w original*.

Modificado (MGS)

para $j = 1, \ldots, k$:

$w = w - (w^T v_j)\, v_j$

Cada coeficiente se calcula con el *w ya corregido*.

En aritmética exacta son idénticos. En punto flotante, MGS mantiene mucho mejor la ortogonalidad.

El ejemplo del apunte

$A = \begin{bmatrix} 1 & 1 & 1 \\ \delta & \delta & 0 \\ \delta & 0 & \delta \end{bmatrix}$:
ortogonalizar sus columnas y medir $\|V^T V - I\|$.

	$\delta = 10^{-7}$		$\delta = 10^{-8}$
CGS	$1.1 \cdot 10^{-3}$	1.0	(v_2, v_3 a 45°)
MGS	$1.6 \cdot 10^{-10}$		$2.0 \cdot 10^{-8}$

Aun con MGS se puede perder ortogonalidad (con A mal condicionada o $k_{\text{máx}}$ grande); el remedio es reortogonalizar. No lo veremos.

GMRes, versión preliminar 2

GMRes (preliminar 2: con Gram–Schmidt modificado)

Dados A, x_0, b, $\mathrm{tol} > 0$, $k_{\text{máx}} \in \mathbb{N}$.

$r_0 = b - Ax_0$; $\quad \beta = \rho = \|r_0\|_2$; $\quad v_1 = r_0/\rho$; $\quad k = 0$;

mientras $\rho > \mathrm{tol}\,\|b\|_2$ **y** $k < k_{\text{máx}}$

$k = k + 1$; $\quad v_{k+1} = Av_k$;

para $j = 1, \ldots, k$

$h_{jk} = v_{k+1}^T v_j$; $\quad v_{k+1} = v_{k+1} - h_{jk} v_j$; % MGS

fin

$h_{k+1,k} = \|v_{k+1}\|_2$; $\quad v_{k+1} = v_{k+1}/h_{k+1,k}$;

$y_k = \operatorname{argmin}_{y \in \mathbb{R}^k} \|\beta e_1 - H_k y\|_2$; % (*)

$\rho = \|\beta e_1 - H_k y_k\|_2$; % (**)

fin

$x_k = x_0 + V_k y_k$.

Lo que falta mejorar

H_{k+1} *contiene* a H_k: se agrega una columna. Rehacer desde cero el problema (*)–(**) en cada iteración es un desperdicio. ¿Se puede actualizar?

5.9.5 Rotaciones de Givens

Recordatorio: rotaciones de Givens (Sección 3.6)

$$G = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}, \qquad c^2 + s^2 = 1.$$

Es ortogonal, y dado $(a, b) \neq (0, 0)$, con

$$\nu = \sqrt{a^2 + b^2}, \qquad c = \frac{a}{\nu}, \qquad s = -\frac{b}{\nu},$$

resulta

$$G \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} (a^2 + b^2)/\nu \\ (-ab + ab)/\nu \end{bmatrix} = \begin{bmatrix} \nu \\ 0 \end{bmatrix}.$$

En n dimensiones

$G_j(c, s)$: la identidad con ese bloque de 2×2 en las filas y columnas $j, j+1$. Multiplicar por G_j solo cambia las filas j y $j+1$: cuesta $O(1)$ por columna.

Para una matriz Hessenberg, una rotación por columna alcanza para llevarla a triangular superior.

Mínimos cuadrados con QR de H_k

Si $H_k = Q_k R_k$, con Q_k ortogonal de $(k+1) \times (k+1)$ y R_k triangular superior de $(k+1) \times k$ (última fila nula), y llamamos $g := Q_k^T \beta e_1$:

$$\|\beta e_1 - H_k y\|_2 = \|Q_k^T(\beta e_1 - H_k y)\|_2 = \|g - R_k y\|_2.$$

$$g - R_k y = \begin{bmatrix} g_{1:k} - (R_k)_{1:k,1:k}\, y \\ g_{k+1} \end{bmatrix}$$

Entonces

- $y_k = (R_k)_{1:k,1:k} \backslash g_{1:k}$ (triangular).
- $\rho = \|r_k\|_2 = |g_{k+1}|$.

Lo importante: para el criterio de parada no hace falta resolver el sistema triangular: alcanza con $|g_{k+1}|$. y se calcula una sola vez, al final.

Actualizar en lugar de rehacer

Si $H_k = Q_k R_k$, como $H_{k+1} = \begin{bmatrix} H_k & h_{1:k+1,k+1} \\ 0^T & h_{k+2,k+1} \end{bmatrix}$:

$$H_{k+1} = \begin{bmatrix} Q_k & 0 \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} R_k & Q_k^T h_{1:k+1,k+1} \\ 0^T & h_{k+2,k+1} \end{bmatrix}.$$

Casi una QR: falta anular un solo elemento, $h_{k+2,k+1}$. Para eso, una rotación nueva G_{k+1} en las filas $k+1, k+2$.

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & \times \\ 0 & 0 & 0 & 0 & \times \end{bmatrix} \xrightarrow{G_5} \begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & \times \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = R_5$$

En azul, la columna nueva *después* de aplicarle las rotaciones viejas $G_1, \ldots, G_4$; en rojo, el elemento a anular.

Como la última fila de R_k es nula, G_{k+1} no toca las columnas viejas.

Qué hay que hacer en cada iteración

1. Calcular la columna nueva $h_{1:k+1,k}$ con Arnoldi (MGS).
2. Aplicarle las rotaciones viejas, en orden:

$$h \leftarrow G_{k-1} \cdots G_2 G_1 \, h \iff h_{j:j+1} \leftarrow G_j \, h_{j:j+1}, \quad j = 1, \ldots, k-1.$$

3. Calcular la rotación nueva G_k que anula $h_{k+1,k}$, y aplicarla a h y a g.
4. $\rho = |g_{k+1}|$.

Qué se guarda

R_k (en el lugar de H_k), los pares $(c_1, s_1), \ldots, (c_k, s_k)$, y el vector g.

Q_k no se forma nunca: $Q_k = (G_k \cdots G_2 G_1)^T$.

Costo de los pasos 2–4: $O(k)$. Lo dominante sigue siendo Arnoldi: $O(nk)$.

GMRes, versión final

GMRes con Gram–Schmidt modificado y rotaciones de Givens

Dados A, x_0, b, $\mathrm{tol} > 0$, $k_{\text{máx}} \in \mathbb{N}$.

$r_0 = b - Ax_0$; $\beta = \rho = \|r_0\|_2$; $v_1 = r_0/\rho$; $g = \beta e_1 \in \mathbb{R}^{k_{\text{máx}}+1}$; $k = 0$;

mientras $\rho > \mathrm{tol}\,\|b\|_2$ **y** $k < k_{\text{máx}}$

- $k = k + 1$; $v_{k+1} = Av_k$;
- **para** $j = 1, \ldots, k$: $h_{jk} = v_{k+1}^T v_j$; $v_{k+1} = v_{k+1} - h_{jk} v_j$; % Arnoldi (MGS)
- $h_{k+1,k} = \|v_{k+1}\|_2$; $v_{k+1} = v_{k+1}/h_{k+1,k}$;
- **para** $j = 1, \ldots, k-1$: $h_{j:j+1,k} = G_j(c_j, s_j)\, h_{j:j+1,k}$; % (†) rotaciones viejas
- $\nu = \sqrt{h_{kk}^2 + h_{k+1,k}^2}$; $c_k = h_{kk}/\nu$; $s_k = -h_{k+1,k}/\nu$; % rotación nueva
- $h_{kk} = \nu$; $h_{k+1,k} = 0$;
- $g_{k:k+1} = G_k(c_k, s_k)\, g_{k:k+1}$;
- $\rho = |g_{k+1}|$; % $= \|b - Ax_k\|_2$, sin calcular x_k

fin

$y = \mathtt{trisup}(H_{1:k,1:k},\, g_{1:k})$; $x_k = x_0 + V_k y$.

- Inicializar $g = \beta e_1$ antes del ciclo: las rotaciones se le aplican a ese vector.
- El sistema triangular se resuelve una sola vez, después del ciclo.

El paso (†) en matlab/octave

```
% aplicar las rotaciones viejas G_1,...,G_{k-1} a la columna nueva
for j = 1:k-1
    G = [c(j) -s(j); s(j) c(j)];
    H(j:j+1,k) = G*H(j:j+1,k);
end
% rotacion nueva: anular H(k+1,k)
nu = hypot(H(k,k), H(k+1,k));
c(k) = H(k,k)/nu;   s(k) = -H(k+1,k)/nu;
H(k,k) = nu;        H(k+1,k) = 0;
g(k:k+1) = [c(k) -s(k); s(k) c(k)] * g(k:k+1);
rho = abs(g(k+1));                          % residuo, gratis
```

`hypot(a,b)` calcula $\sqrt{a^2 + b^2}$ evitando overflow y underflow. Verificado: $\rho = |g_{k+1}|$ coincide con $\|b - Ax_k\|_2$ a precisión de máquina.

Costo por iteración: GC vs. GMRes

	GC (A sdp)	**GMRes** (iteración k)
Productos mat-vec	1	1
Productos escalares	2	$k+1$ (MGS + norma)
Operaciones *axpy*	3	k
Rotaciones de Givens	—	k, de 2×2: $O(k)$
Memoria	4 vectores	$k+1$ vectores, más $O(k^2)$ para H
Total hasta la iteración k	$O(nk)$	$O(nk^2)$

La moraleja

La simetría da la recurrencia corta; sin ella, cada iteración es más cara que la anterior. Por eso en GMRes es todavía más importante precondicionar bien: pocas iteraciones son la única manera de mantener acotados la memoria y el costo.

Números

Restart: cuándo duele y cuándo no

Ejemplo con convección de la clase pasada: $A = A_h + 10\,C$, $n = 255$, $b = (1, \ldots, 1)^T$, tolerancia 10^{-8}. Iteraciones totales (mat-vecs) de GMRes(m):

	$m = 5$	$m = 10$	$m = 20$	sin restart
Sin precondicionar	> 5000 (no converge)	3845	1856	255
Precondicionado ($M = A_h^{-1}$, a derecha)	34	21	17	17

- **Sin precondicionar**, el restart es catastrófico: con $m = 5$ GMRes se estanca, y con $m = 20$ necesita siete veces más productos matriz–vector que GMRes completo. (Y GMRes completo, a su vez, necesita guardar 255 vectores.)
- **Con un buen precondicionador**, el restart casi no cuesta nada: con $m = 20$ el resultado es el mismo que sin restart, guardando 21 vectores.

Precondicionar bien es lo que hace viable a GMRes(m) con m chico.

Para hacer

Problemas del capítulo

- **5.15:** programar GMRes con rotaciones de Givens, lo más eficiente posible.
  Test: verificar que $|g_{k+1}|$ coincide con $\|b - Ax_k\|_2$.
- **5.16:** programar GMRes con restart, GMRes(m).
  Reproducir la tabla anterior.
- **5.17:** comparar GC y GMRes en el sistema del Problema 5.5.
  ¿Qué se espera, si A es sdp?
- **5.18:** el problema con convección: GMRes, CGNE y CGNR.

Del texto

Reproducir el ejemplo $\delta = 10^{-7}$ de Gram–Schmidt clásico vs. modificado, y probar con $\delta = 10^{-8}$.

Para recordar

1. **Arnoldi** = Gram–Schmidt sobre $\mathcal{K}_k$, ortogonalizando Av_k. Da $AV_k = V_{k+1}H_k$, con H_k Hessenberg de $(k+1) \times k$.
2. **El problema chico:** $\|r_k\|_2 = \|\beta e_1 - H_k y\|_2$. De $n \times k$ a $(k+1) \times k$, y el residuo se conoce sin calcular x_k.
3. **Breakdown** ($z_{i+1} = 0$): GMRes ya encontró la solución exacta.
4. **Gram–Schmidt modificado**: idéntico en aritmética exacta, mucho más estable en punto flotante.
5. **Givens:** una rotación nueva por iteración actualiza la QR de H_k; $\rho = |g_{k+1}|$ sale gratis, y el sistema triangular se resuelve una vez, al final.
6. **Costo:** $O(nk)$ en la iteración k, $O(nk^2)$ en total, y $k+1$ vectores en memoria. No hay recurrencia corta.
7. **Restart:** memoria fija, sin teoría de convergencia; viable cuando el precondicionador es bueno.
8. **Cierre del Capítulo 5:** métodos de Krylov para sdp (GC) y no simétricos (GMRes), y el precondicionamiento como la pieza que los hace eficientes.